

JSON Schema Nedir?

Veriyi tanımlayan kural seti — bir sözleşme gibi

Gerçek Hayat Örneği

Bir sipariş formu düşünün:

- ✦ Ad → boş bırakılamaz
- ✦ Yaş → sayı, 0'dan büyük
- ✦ E-posta → @ içermeli
- ✦ Ülke → listeden seçilmeli

JSON Schema bunu kod olarak tanımlar. Validator ise gelen veriyi bu kurallara göre kontrol eder — otomatik, hızlı, güvenli.

Schema (kural tanımı)

```
{
  "type": "object",
  "required": ["ad", "yas"],
  "properties": {
    "ad": {
      "type": "string",
      "minLength": 1
    },
    "yas": {
      "type": "integer",
      "minimum": 0
    }
  }
}
```

Sonuç

✓ Geçerli veri

```
{"ad": "Mert",
  "yas": 26}
```

→ `validate() = true`

✗ Geçersiz veri

```
{"ad": "",
  "yas": -5}
```

→ ad boş olamaz (minLength: 1)

→ yas negatif olamaz (minimum: 0)

→ **`validate() = false`**

→ 2 hata detayı döner

Kullanım alanları:

API Gateway

Her request otomatik doğrulama

Form Validasyon

Sunucuya varmadan kontrol

Veritabanı

Kayıt öncesi kural kontrolü

Mikroservis

Servisler arası sözleşme

ata-validator vs ajv — Neden Fark Büyük?

ajv — JavaScript

Schema Derleme

831 ops/sn

Her kullanımda yeniden derlenir

Regex Güvenliği

std::regex — backtracking

$O(2^n)$ → ReDoS saldırısına açık

Bellek Kullanımı

Tüm JSON DOM ağacına dönüştürülür

Heap allocasyon, GC baskısı

Validasyon Hızı

JS engine sınırları

~yüzlerce K ops/sn

ata-validator — Native C++

Schema Derleme

155.000 ops/sn

137x daha hızlı — bir kez derle, hep kullan

Regex Güvenliği

Google RE2 — NFA simulation

$O(n)$ garanti → ReDoS imkânsız

Bellek Kullanımı

On Demand API — DOM oluşturulmaz

Sıfır allocasyon, GC yok

Validasyon Hızı

Native C++ hızı

1.400.000 ops/sn

API Uyumluluğu — drop-in replacement:

```
// ajv
const ajv = new Ajv();
ajv.validate(schema, data);
```

```
// ata-validator
const {Validator} = require("ata-validator");
new Validator(schema).validate(data);
```

Geleneksel Yöntem — Bayt Bayt

JSON: { "ad": 26 }

Her bayt ayrı ayrı işlenir:



CPU döngüleri:



Her adımda:

1. Baytı register'a yükle
2. Kontrol et: { mi? mi? : mi?
3. Dal (branch) — if/else
4. Sonraki bayta geç

Maliyet: $O(n)$ adım $\times$ skaler işlem

Dal tahmini başarısız $\rightarrow$ CPU pipeline durur

simdjson — SIMD (256-bit vektör)

JSON: { "ad": 26, ... (32 bayt)

32 bayt tek register'a yüklenir:

256-bit SIMD register $\rightarrow$ 32 bayt aynı anda

↑ 1 CPU talimatı ↑

Paralel soru:

Hangi pozisyonlar { } ?

$\rightarrow$ structural chars

Hangi pozisyonlar quotes?

$\rightarrow$ string sınırları

Hangi pozisyonlar : , ?

$\rightarrow$ ayraçlar

Kazanım: 4+ GB/sn parse hızı

x86 AVX2 / ARM NEON — donanım hızında

ata-validator

JSON Schema'yı neden
yeniden yazmak gerektiği?

Bilimsel Motivasyon
& Mühendislik Kararları

01 Hız Sorunu

JSON validasyonu, API gateway'lerde ve yüksek frekanslı sistemlerde dar boğaz.

02 Güvenlik Açığı

std::regex ile yazılan pattern validatörleri ReDoS saldırısına karşı savunmasız.

03 Spesifikasyon Boşluğu

Çoğu C++ validator Draft 2020-12'yi desteklemiyor; ecosystem JavaScript'te.

04 Bellek Verimsizliği

DOM-based ayrıştırıcılar gereksiz heap allocasyonu yapıyor, GC baskısı yaratıyor.

Bilimsel Temel I — SIMD Parsing

simdjson: Daniel Lemire et al., 2019 — VLDB Best Paper

Geleneksel JSON Ayırıştırma

Bayt bayt okuma

Her karakter için dal (branch) talimatı

Skaler karşılaştırma

Tek bayt işlem, CPU pipeline boş kalır

Sıralı yapı çözme

Derinlik artıkça yığın büyür

Tampon kopyalama

String değerleri heap'e kopyalanır

simdjson: SIMD-Accelerated

256-bit vektör işlem

32 baytı tek CPU talimatında işle

Stage 1: Structural scan

{ } [] , : " konumlarını bul

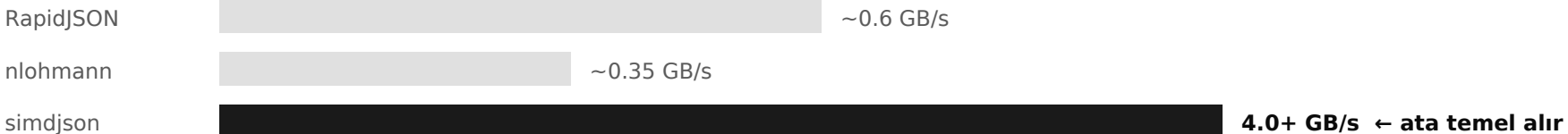
Stage 2: DOM/On-Demand

Sadece erişilen değerleri çöz

Sıfır kopya string

Kaynak tamponuna pointer döndür

Ölçülen verim (Apple M-series):



Bilimsel Temel II — RE2 & ReDoS

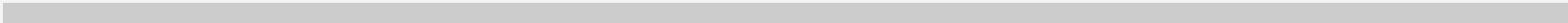
Cox, 2007 — 'Regular Expression Matching Can Be Simple And Fast'

ReDoS Nedir? (Regular Expression Denial of Service)

Backtracking tabanlı regex motorları bazı girdilerde üstel (exponential) zamanda çalışır. Tek bir kötü niyetli JSON string değeri, sunucuyu bloke edebilir.

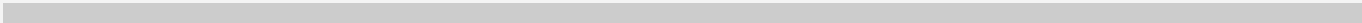
Zaman Karmaşıklığı Karşılaştırması

`std::regex` (backtracking)



$O(2^n)$ — worst case

Python re



$O(2^n)$ — worst case

RE2 (NFA simulation)



$O(n)$ — her zaman

RE2'nin Teknik Dayanağı

NFA (Nondeterministic FA)

- Regex → NFA dönüşümü: Thompson'ın yapımı, $O(n)$ zaman & alan
- Paralel durum simülasyonu
- Backtracking yok → garantili $O(n \cdot m)$

ata'da RE2 kullanımı

- pattern: anahtar kelimesi RE2'ye yönlendir
- format: el yazısı parser (regex bile yok)
email, uri, ipv4, uuid, date, hostname
- Compile-time regex cache: tekrar derleme yok

Mühendislik Kararı — Bytecode & On Demand

İki temel optimizasyon: compile-time bytecode + single-pass validation

Bytecode Compiler

Schema bir kez derlenir; bytecode defalarca çalıştırılır.
Ağaç yürüme (tree-walk) yerine düz talimat dizisi.

```
// bytecode plan
EXPECT_OBJECT
CHECK_REQUIRED      "name"
CHECK_REQUIRED      "email"
OBJ_PROPS_START
  CHECK_NO_ADDITIONAL
  OBJ_PROP  "name"  → sub[0]
  OBJ_PROP  "email" → sub[1]
OBJ_PROPS_END
END

sub[0]: EXPECT_STRING, CHECK_MIN_LENGTH 1
sub[1]: EXPECT_STRING, CHECK_FORMAT email
```

✓ Sıfır allocasyon

Hot path'te heap erişimi yok

✓ Branch azaltma

if/else yerine opcode dispatch

✓ Önbellek dostu

Düz dizi L1 cache'e sığar

On Demand API

Uygun şemalar için DOM hiç oluşturulmaz.
JSON token akışı doğrudan doğrulanır.

Geleneksel DOM yolu

JSON string

→ full parse + tree build

→ DOM tree (heap alloc)

→ tree walk

→ result

ata On Demand yolu

JSON string

→ streaming tokenize

→ validate inline

→ result

(DOM yok. Allocasyon yok.)

2.3x daha hızlı

5 KB dokümanda On Demand vs DOM (70K vs 31K ops/sn)

Benchmark Sonuçları

Apple Silicon M-series · Pre-compiled schema · Gerçekçi iç içe JSON belgeleri

Schema Compile — ata vs ajv

ata: 137x daha hızlı schema derleme

 155K ops/sn

 ajv/DOM: 831 ops/sn

137x

Geçerli belge validasyonu

Pre-compiled schema, native C++ hızı

 1.4M ops/sn

1.4M

Geçersiz belge validasyonu

Erken çıkış optimizasyonu

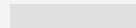
 730K ops/sn

730K

On Demand — 5 KB belge

DOM oluşturmadan 2.3x hız kazancı

 70K ops/sn

 ajv/DOM: 31K ops/sn

2.3x

Basit tip kontrolü

En yalın case — SIMD hızı tam devrede

 13.8M ops/sn

13.8M

JSON Schema Draft 2020-12 Test Suite

939 / 952 test geçti

Resmi spec test suite

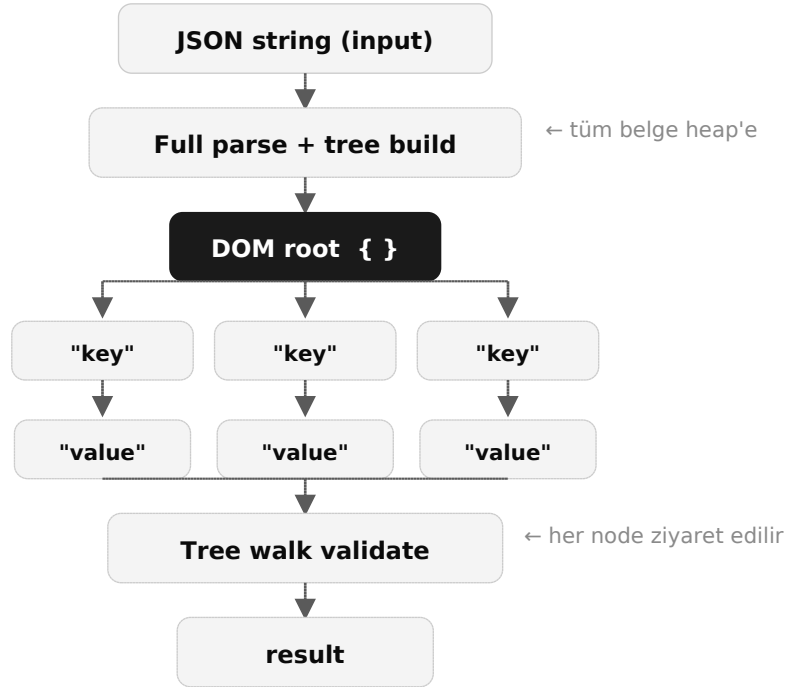
98.6% uyum oranı

Draft 2020-12

Eksik: `unevaluatedProperties`, `unevaluatedItems`

Edge-case özellikler

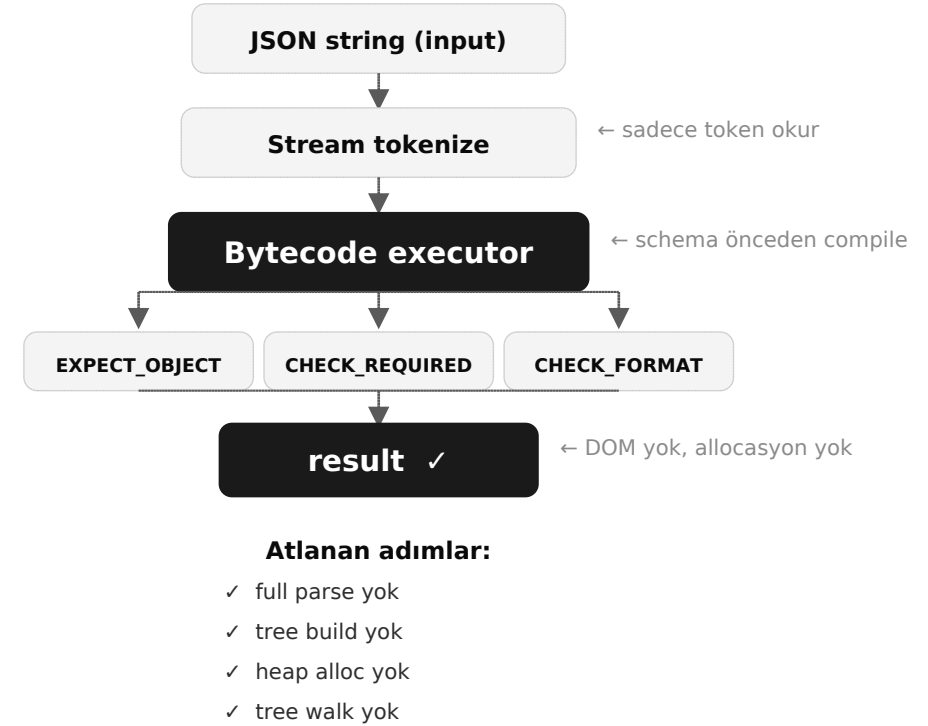
AS-IS — DOM Tabanlı (geleneksel)



Maliyet:

- ✗ Tüm belge parse → heap allocasyon
- ✗ DOM ağacı oluşturma → ekstra bellek
- ✗ Her node ziyareti → $O(n)$ tree walk

NEW — ata On Demand (tek geçiş)



Kazanım:

- ✓ DOM allocasyonu yok → daha az bellek
- ✓ Tek geçiş → 2.3x hız (5 KB belgede)
- ✓ Bytecode executor → sıfır hot-path alloc

Neden ata-validator? — Özet

Her tasarım kararının arkasında ölçülebilir bir bilimsel gerekçe var.

01 simdjson

SIMD talimatları ile
4+ GB/s parse hızı.
Lemire 2019 VLDB Best Paper.

02 RE2

Thompson NFA: $O(n)$ garantisi.
ReDoS saldırısı imkânsız.
format: sıfır regex.

03 Bytecode

Compile-time schema $\rightarrow$ opcode.
Hot path'te sıfır allocasyon.
L1 cache dostu.

04 On Demand

DOM oluşturmadan validate.
Tek geçiş, 2.3x hız kazancı.
Büyük belgeler için ideal.

github.com/mertcanaltin/ata-validator

MIT Lisansı — Katkıya açık

`npm install ata-validator`

137x

ajv'den hızlı
compile

1.4M

validasyon/sn

98.6%

spec
uyumu